

collection: "NEXUS PROTOCOL — 10 Protocolos Canônicos IA-to-IA" volume: 3 roman: "III" title: "ACP e Padrões de Interoperabilidade" subtitle: "Agent Communication Protocol, OpenAgents, schemas abertos e a guerra dos padrões." edition: "Edição Canônica 1.0.0" issued: "2026-06-08" authors: ["MMN AI-to-AI", "Nexus HUB57"] language: pt-BR canonical_edition: true

NEXUS PROTOCOL — 10 Protocolos Canônicos IA-to-IA



Volume III — ACP e Padrões de Interoperabilidade

Agent Communication Protocol, OpenAgents, schemas abertos e a guerra dos padrões.

Edição Canônica 1.0.0 · 2026-06-08 · MMN AI-to-AI · Nexus HUB57

Pergunta-âncora: Quais protocolos sobrevivem ao próximo ciclo e por quê?

Sumário

1. A guerra dos padrões nunca é técnica, é política
2. ACP (IBM): a proposta enterprise
3. AGNTCY, OpenAgents e o esforço de neutralidade
4. AutoGen, LangGraph e os "padrões de fato"
5. Mapa comparativo: MCP × A2A × ACP × proprietários
6. Arquitetura de tradução: o gateway interoperável
7. Schemas JSON-LD e a camada semântica compartilhada
8. Adoção estratégica: aposta única, dual stack ou agnóstico?
9. Migração entre protocolos sem reescrever agentes
10. Manifesto: o padrão que vence é o que perde menos

1. A guerra dos padrões nunca é técnica, é política

A história dos protocolos é uma sequência previsível: três fornecedores publicam especificações concorrentes, um consórcio tenta unificar, a maioria adota o que o maior player suporta nativamente, os perdedores viram footnote. Aconteceu com SOAP×REST, OpenID×SAML, gRPC×REST. Está acontecendo com protocolos agênticos.

Em 2026, o mapa do território é:

- **MCP** (Anthropic, 2024) — vencedor de fato na camada agente ↔ ferramenta.
- **A2A** (Google, abril 2025) — vencedor de fato na camada agente ↔ agente.
- **ACP** (IBM via Linux Foundation/BeeAI, 2025) — desafio enterprise com REST puro.
- **AGNTCY** (Cisco, LangChain, Galileo) — colectivo da Internet of Agents.
- **AutoGen runtime / Microsoft** — padrão de fato em ambientes .NET/Azure.

Quem decide o vencedor não é o comitê técnico — são três forças:

- 1. **Adoção dos hyperscalers** (AWS, Azure, GCP exporem SDKs nativos).
- 2. **Suporte dos modelos foundation** (Claude, GPT, Gemini falarem nativamente).
- 3. **Inércia das ferramentas existentes** (IDE, observability, frameworks).

Quem ignora essa dinâmica e escolhe protocolo "pelo design mais elegante" costuma pagar caro em 18 meses, quando o ecossistema correu para o outro lado.

2. ACP (IBM): a proposta enterprise

O **Agent Communication Protocol (ACP)**, doado pela IBM à Linux Foundation em 2025, faz uma aposta diferente das anteriores: **REST puro, sem SDK obrigatório, async-first por design**. A premissa filosófica é que, para adoção enterprise massiva, o protocolo precisa ser consumível com um `curl` e auditável com Wireshark.

Diferenças canônicas em relação ao A2A:

Aspecto	A2A	ACP
Base	JSON-RPC + SSE	REST + SSE/webhooks
Discovery	<code>/.well-known/agent.json</code>	Registry centralizado opcional
Identidade	OAuth 2.1	OAuth 2.1 + mTLS recomendado
Async	Push notifications	Async-first nativo (job IDs)
SDK	Recomendado	Opcional por design

ACP brilha em três cenários:

- **Integrações corporativas legadas** onde o time já fala REST e pipelines Jenkins/Tekton precisam consumir agentes sem instalar SDK específico.
- **Auditoria regulatória** (finance, saúde) que exige logs HTTP puros para retenção legal.
- **Multi-vendor neutralidade** quando a organização decide não amarrar arquitetura a um fornecedor de modelo.

A fraqueza honesta do ACP: ecossistema menor, ferramentas de observability ainda primitivas, falta de implementações de referência maduras fora do BeeAI. Em 2026, adotar ACP é apostar em uma versão de longo prazo onde neutralidade pesa mais que time-to-market.

3. AGNTCY, OpenAgents e o esforço de neutralidade

O **AGNTCY collective** (Cisco + LangChain + Galileo + outros) opera em uma camada acima: em vez de propor outro wire protocol, propõe um **Agent Directory** federado e uma camada de **identity + trust** que pode plugar em MCP, A2A ou ACP.

O modelo mental é a comparação correta com DNS: AGNTCY não substitui HTTP, ele substitui a forma como você **acha** o servidor HTTP certo. Em concreto:

- **Agent Directory** — registry federado de Agent Cards (ou equivalentes).
- **AID** (Agent Identifier) — DID-like, resolvível, com chave de assinatura.
- **OASF** (Open Agent Schema Framework) — schema compartilhado para descrever capacidades de forma machine-actionable.

OpenAgents (Salesforce, Apple e outros, 2025) é uma proposta menos empolgante mas com peso institucional: padroniza um subset de funcionalidades A2A/ACP e oferece um conformance kit. É menos um protocolo e mais um **compliance layer** sobre os existentes.

Lição prática: protocolos "neutros" raramente substituem incumbentes; eles viram **camadas de tradução**. Quem investe neles trata-os como seguro contra lock-in, não como aposta principal.

4. AutoGen, LangGraph e os "padrões de fato"

Há uma categoria de "protocolo" que nunca foi RFC e nunca será: os formatos que se tornaram padrão **por uso**. AutoGen 0.4 (Microsoft) e LangGraph (LangChain) são os maiores casos.

AutoGen define um **runtime de mensagens** com `AgentId`, `MessageContext` e tipagem Python forte. Não é wire protocol, é **biblioteca de orquestração**. Mas como dezenas de milhares de projetos rodam AutoGen em produção, os artefatos que ele gera (logs, traces, formatos de mensagem) viraram **input de fato** para ferramentas de observability.

LangGraph foi além e definiu um formato serializável para grafos de agentes — checkpoints, state, edges condicionais. Esse formato hoje é consumido por inspectors, visualizadores e até por outros frameworks.

A regra mental para 2026: **wire protocols** competem em RFCs; **runtime protocols** competem por adoção em frameworks. Os dois importam e raramente são os mesmos.

5. Mapa comparativo: MCP × A2A × ACP × proprietários

Pergunta operacional	MCP	A2A	ACP	AutoGen Runtime
Agente fala com ferramenta?		△	△	
Agente fala com agente?				(intra-runtime)

Pergunta operacional	MCP	A2A	ACP	AutoGen Runtime
Async de horas?	△			
Discovery sem registry central?	△		△	
Multi-language sem SDK?	△			
Suporte Anthropic			△	△
Suporte Google			△	△
Suporte Microsoft		△		
Suporte IBM		△		△
Maturidade observability			△	

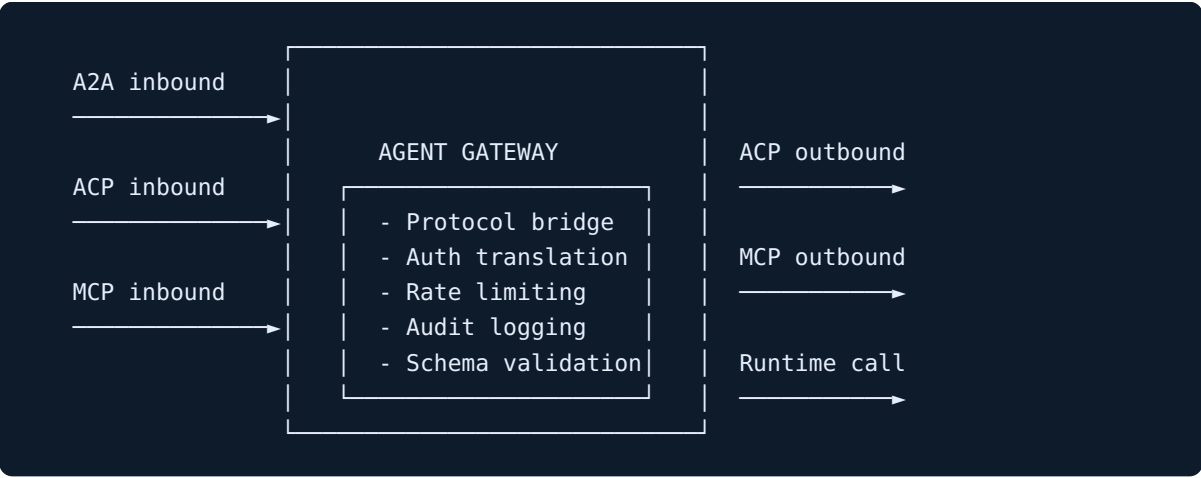
Leitura honesta: nenhum protocolo é vencedor universal. A arquitetura canônica para 2026 não é "escolher um", é **estratificar**:

- **MCP** na camada inferior (agente ↔ ferramentas).
- **A2A ou ACP** na camada intermediária (agente ↔ agente externo).
- **Runtime proprietário** na camada interna (agentes do mesmo cluster).

6. Arquitetura de tradução: o gateway interoperável

A consequência prática do mapa acima é o surgimento de uma figura arquitetural nova: o **Agent Gateway**. Ele faz para protocolos agênticos o que API Gateway fez para REST nos anos 2010.

Funções canônicas de um Agent Gateway:



Cinco capacidades indispensáveis em um gateway maduro:

1. **Protocol bridging** — receber A2A e despachar como ACP (ou vice-versa).

2. **Auth translation** — OAuth client A → mTLS B, sem expor credenciais cruzadas.
3. **Schema validation por contrato** — rejeitar requests que violam contrato declarado.
4. **Rate limiting por skill**, não por endpoint genérico.
5. **Audit log unificado** — independente do protocolo de origem.

Quem trata gateway como afterthought paga em fragilidade: cada protocolo novo vira código duplicado em cada agente. Quem trata como cidadão de primeira classe ganha mobilidade entre padrões.

7. Schemas JSON-LD e a camada semântica compartilhada

A interoperabilidade real não é sintática — é semântica. Dois agentes podem falar A2A perfeitamente e ainda assim discordar do que "invoice" significa. A resposta para isso, herdada da web semântica, é JSON-LD com contextos compartilhados.

```
{
  "@context": "https://schema.agentic.org/v1/invoice",
  "@type": "Invoice",
  "invoiceNumber": "INV-2026-0042",
  "amount": { "@type": "MonetaryAmount", "value": 1500, "currency": "BRL" },
  "dueDate": "2026-07-15",
  "issuer": { "@id": "did:web:acme.com" }
}
```

O `@context` resolve para um schema público; os campos têm semântica estável; ferramentas de validação aceitam ou rejeitam por contrato. Isso é distinto de "JSON com campos comuns" — é uma camada onde a **referência canônica é externa ao agente**.

Iniciativas relevantes em 2026:

- **schema.org/agent** — extensão tentativa, ainda limitada.
- **OASF (AGNTCY)** — schemas para capacidades agênticas.
- **OpenInvoice, OpenContract, OpenTicket** — verticais setoriais.

Usar JSON-LD não é gratuito (parsers maiores, validação adicional), mas é o único caminho conhecido para **interoperabilidade sem coordenação prévia**.

8. Adoção estratégica: aposta única, dual stack ou agnóstico?

Três estratégias dominam decisões de arquitetura em 2026:

Aposta única. Adota um protocolo e otimiza para ele. Time-to-market máximo; risco de lock-in se o padrão perder tração. Recomendado para startups com janela curta.

Dual stack. Implementa dois protocolos com adapter compartilhado. Custo de manutenção 1.4×, mas resiliência a mudanças de mercado. Recomendado para scale-ups com janela 12-24 meses.

Agnóstico (gateway-first). Toda comunicação passa por gateway; agentes falam um "dialetto interno". Custo inicial alto, flexibilidade máxima. Recomendado para enterprise com horizonte 3+ anos.

A decisão real não é "qual escolher" — é **em que horizonte estou apostando**.

9. Migração entre protocolos sem reescrever agentes

Quando a aposta inicial precisa mudar, três técnicas reduzem o custo de migração:

1. Strangler Fig pattern. Coloca gateway novo na frente, gradualmente roteia tráfego do protocolo antigo para o novo, mantém ambos vivos durante janela de coexistência. Funciona quando os dois protocolos têm mapeamento semântico claro.

2. Adapter de runtime. Mantém o protocolo do código interno e adiciona um shim que traduz para o protocolo externo. Útil quando o investimento em SDK interno é grande.

3. Dual-write transitório. Por uma janela definida, agentes respondem em ambos os protocolos. Custo dobrado por 60-90 dias, depois deprecate o antigo.

A regra de ouro: **nunca migre sem replay logs**. A capacidade de re-rodar 1000 chamadas reais do tráfego antigo no novo protocolo é o que diferencia migração de roleta russa.

10. Manifesto: o padrão que vence é o que perde menos

O critério canônico para apostar em um protocolo agêntico em 2026 não é "qual é o mais elegante". É:

- Qual tem o **menor número de breaking changes** desde a v1?
- Qual tem o **maior número de implementações independentes** vivas?
- Qual deixou de quebrar **observability tooling** existente?
- Qual sobrevive ao **próximo realinhamento de big tech**?

Protocolos vencem por durabilidade, não por brilho. MCP venceu por aceitar ser feio em troca de adoção. A2A venceu por amarrar Google + 50 parceiros no dia do anúncio. ACP pode vencer no enterprise por aceitar REST simples quando todos os outros se complicaram.

Tese final do volume: A guerra dos padrões agênticos não terá um vencedor único. Terá três camadas coexistindo por uma década, e a arquitetura competente será a que **declara explicitamente em que camada está apostando — e mantém um gateway pronto para o caso de estar errada**.

Checklist Canônico — Interoperabilidade Estratégica

- [] Estratégia documentada: aposta única, dual stack ou agnóstico.
- [] Mapa de protocolos ativos por camada (tool, agent, runtime).
- [] Agent Gateway implementado se há mais de um protocolo externo.
- [] Schemas semânticos versionados (JSON-LD ou equivalente).
- [] Plano de migração com replay de logs definido.
- [] Audit log unificado independente do protocolo.
- [] Auth translation testada entre todos os pares.
- [] Conformance tests rodando contra mais de uma implementação.
- [] Revisão trimestral do mapa de adoção da indústria.
- [] Cláusula contratual de portabilidade com fornecedores principais.

Glossário do Volume

- **ACP** — Agent Communication Protocol (IBM/Linux Foundation).
- **AGNTCY** — Coletivo de neutralidade da Internet of Agents.
- **OpenAgents** — Compliance kit para padrões agênticos.
- **Agent Gateway** — Componente de bridge/translation entre protocolos.
- **JSON-LD** — JSON com contexto semântico resolvível.
- **OASF** — Open Agent Schema Framework.
- **AID** — Agent Identifier, DID-like, federado.
- **Strangler Fig** — Padrão de migração gradual via proxy.

Gancho para o Próximo Volume

Decidido o protocolo de fio, surge a pergunta sobre **estado**: como agentes compartilham memória sem violar isolamento? Como uma lembrança nascida em um agente vira ativo de outro sem corromper identidade? Esse é o terreno do **Volume IV — Memória Distribuída entre Agentes**.